

Helping a GrandPa! — Documentación Completa

Trabajo Fin de Grado

Autor: Carlos Alfaro García **Titulación:** Desarrollo de Aplicaciones Multiplataforma

Índice

- [1. Resumen del Proyecto](#)
- [2. Arquitectura General](#)
- [3. Tecnologías Utilizadas](#)
- [4. Estructura del Proyecto](#)
- [5. Modelos de Datos \(Room Entities\)](#)
- [6. Acceso a Datos \(DAOs\)](#)
- [7. Base de Datos \(Room Database\)](#)
- [8. Actividades \(Pantallas\)](#)
- [9. Adaptadores \(RecyclerView\)](#)
- [10. Helpers y Utilidades](#)
- [11. Firebase](#)
- [12. Servicios en Segundo Plano](#)
- [13. UI y Accesibilidad](#)
- [14. Tests](#)
- [15. Landing Page](#)
- [16. Diagrama de Interacciones](#)

1. Resumen del Proyecto

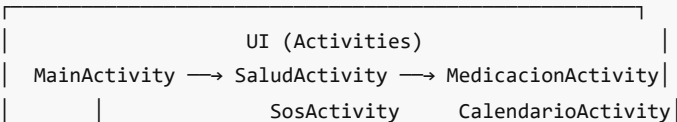
Helping a GrandPa! es una aplicación Android diseñada para ayudar a personas mayores a gestionar su salud y bienestar, así como a sus familiares y cuidadores a estar informados y poder reaccionar ante emergencias.

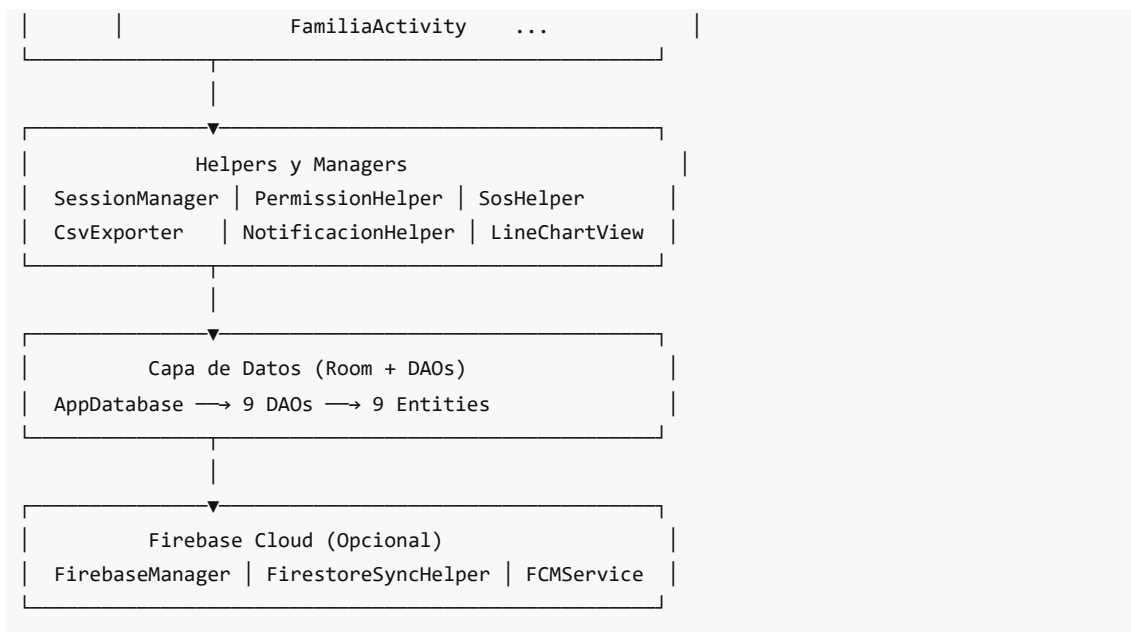
Funcionalidades principales

- Recordatorio de medicación** — Programación de medicamentos con dosis y frecuencia, notificaciones fiables incluso tras reinicio del dispositivo (WorkManager).
- SOS real con ubicación** — Botón de emergencia que envía SMS + WhatsApp + llamada telefónica con localización GPS exacta a contactos de emergencia.
- Seguimiento de salud** — Registro de presión arterial, glucosa, peso y estado de ánimo con gráficas interactivas y exportación CSV.
- Calendario compartido** — Eventos y citas sincronizados con Firebase para visibilidad familiar.
- Círculo familiar** — Miembros con roles (admin, caregiver, viewer) y sincronización cloud.

2. Arquitectura General

La aplicación sigue una arquitectura **offline-first** con persistencia local en Room y sincronización opcional con Firebase Cloud Firestore.





Principios de diseño

- **Offline-first**: Room como fuente de verdad principal. Firebase actúa como capa de sincronización.
- **Reactivo**: Uso de Kotlin Flow para observar cambios en la base de datos.
- **Corrutinas**: Operaciones asíncronas con corrutinas de Kotlin.
- **WorkManager**: Para trabajos en segundo plano que deben sobrevivir al cierre de la app y reinicios.
- **Material 3**: Interfaz adaptada con Material Design 3 y splash screen.

3. Tecnologías Utilizadas

Tecnología	Versión	Propósito
Kotlin	2.0.21	Lenguaje principal
Android Gradle Plugin	8.13.1	Compilación
minSdk	27	Compatibilidad (Android 8.1)
targetSdk	36	Último SDK objetivo
Room	2.6.1	Base de datos local
WorkManager	2.10.0	Trabajos en segundo plano
Firebase BoM	33.12.0	Plataforma Firebase
Firebase Auth	—	Autenticación cloud
Firestore	—	Base de datos cloud
FCM	—	Notificaciones push
Google Location	21.3.0	Servicios de localización
Material 3	—	Componentes UI

SplashScreen	1.0.1	Pantalla de inicio
ViewPager2	—	Onboarding

4. Estructura del Proyecto

```
app/
├─ src/
│  └─ main/
│     ├── java/com/example/miaplicacion/
│     │  ├── *.kt                → 62 archivos Kotlin
│     │  └─ res/
│     │     ├── layout/          → 32 layouts XML
│     │     ├── drawable/        → Iconos vectoriales + imágenes
│     │     ├── values/          → Temas, colores, strings
│     │     ├── xml/             → FileProvider paths
│     │     └─ anim/             → Animaciones de transición
│     ├── AndroidManifest.xml
│     └─ google-services.json    → Configuración Firebase
├─ test/                        → Tests unitarios (10)
└─ androidTest/                 → Tests instrumentados (10)
├─ build.gradle.kts
└─ google-services.json
```

5. Modelos de Datos (Room Entities)

5.1 UserEntity (UserEntity.kt)

Tabla usuarios — Usuarios de la aplicación.

Campo	Tipo	Descripción
id	Int (PK, autoGenerate)	Identificador único
nombre	String	Nombre del usuario
apellidos	String	Apellidos
email	String	Correo electrónico
password	String	Contraseña
fechaNacimiento	String	Fecha de nacimiento
rol	String	Rol (usuario, cuidador, profesional)
telefono	String	Teléfono
direccion	String	Dirección

5.2 Medicacion (Medicacion.kt)

Tabla medicaciones — Medicamentos programados.

Campo	Tipo	Descripción
id	Int (PK)	Identificador
nombre	String	Nombre del medicamento
dosis	String	Dosis (ej: 1 pastilla)
frecuencia	String	Frecuencia (ej: cada 8h)
hora	String	Hora de la toma (HH:mm)
activo	Boolean	Si está activo
userId	Int	FK al usuario

5.3 ContactoEmergencia (ContactoEmergencia.kt)

Tabla contactos_emergencia — Contactos para el sistema SOS.

Campo	Tipo	Descripción
id	Int (PK)	Identificador
nombre	String	Nombre del contacto
telefono	String	Teléfono
email	String	Email
userId	Int	FK al usuario

5.4 RegistroSalud (RegistroSalud.kt)

Tabla registros_salud — Mediciones de salud.

Campo	Tipo	Descripción
id	Int (PK)	Identificador
tipo	String	Tipo (presion, glucosa, peso, animo)
valor	String	Valor medido
nota	String	Nota opcional
fecha	String	Fecha del registro
userId	Int	FK al usuario

5.5 EventoCalendario (EventoCalendario.kt)

Tabla eventos_calendario — Eventos del calendario.

Campo	Tipo	Descripción
id	Int (PK)	Identificador
titulo	String	Título del evento
descripcion	String	Descripción
fecha	String	Fecha (dd/MM/yyyy)
hora	String	Hora (HH:mm)
tipo	String	Tipo (personal, medica, familiar, social)
userId	Int	FK al usuario
firestoreId	String	ID en Firestore para sincronización

5.6 MiembroFamiliar (MiembroFamiliar.kt)

Tabla `miembros_familia` — Miembros del círculo familiar.

Campo	Tipo	Descripción
id	Int (PK)	Identificador
nombre	String	Nombre del miembro
email	String	Email
telefono	String	Teléfono
rol	String	Rol (viewer, caregiver, admin)
mayorId	Int	ID del mayor asociado
userId	Int	FK al usuario
firestoreId	String	ID en Firestore

5.7 Abuelito (Abuelito.kt)

Tabla `abuelitos` — Personas mayores registradas.

Campo	Tipo	Descripción
id	Int (PK)	Identificador
nombre	String	Nombre
apellidos	String	Apellidos
edad	Int	Edad
direccion	String	Dirección
telefonoContacto	String	Teléfono de contacto

necesidades	String	Necesidades especiales
userId	Int	FK al usuario que lo registró

5.8 ServicioHistorial (ServicioHistorial.kt)

Tabla `servicios_historial` — Historial de servicios solicitados.

Campo	Tipo	Descripción
id	Int (PK)	Identificador
horasMedicas	String	Horas de citas médicas
fechaMedicas	String	Fecha de citas médicas
horasCompras	String	Horas de compras
fechaCompras	String	Fecha de compras
totalHoras	String	Total horas
totalPrecio	String	Precio total (3.99€/h)
fechaSolicitud	String	Fecha de solicitud
userId	Int	FK al usuario

5.9 Trabajador (Trabajador.kt)

Tabla `trabajadores` — Trabajadores/cuidadores registrados.

Campo	Tipo	Descripción
idTrabajador	Int (PK)	Identificador
nombre	String	Nombre
apellido	String	Apellido
dni	String	DNI/NIE
email	String	Email
telefono	String	Teléfono

6. Acceso a Datos (DAOs)

6.1 UserDao (UserDao.kt)

- `getAll()` → `Flow<List<UserEntity>>`
- `login(email, password)` → `UserEntity?` (suspend)
- `getByEmail(email)` → `UserEntity?` (suspend)
- `getById(id)` → `UserEntity?` (suspend)
- `insert(user)` → `Long` (suspend)

- `update(user)` (suspend)
- `delete(user)` (suspend)

6.2 MedicacionDao (MedicacionDao.kt)

- `getByUserId(userId)` → `Flow<List<Medicacion>>`
- `getById(id)` → `Medicacion?` (suspend)
- `insert(medicacion)` → `Long` (suspend)
- `update(medicacion)` (suspend)
- `delete(medicacion)` (suspend)

6.3 ContactoEmergenciaDao (ContactoEmergenciaDao.kt)

- `getByUserId(userId)` → `Flow<List<ContactoEmergencia>>`
- `getById(id)` → `ContactoEmergencia?` (suspend)
- `getAllByUserId(userId)` → `List<ContactoEmergencia>` (suspend) — Usado por SOS
- `insert(contacto)` → `Long` (suspend)
- `update(contacto)` (suspend)
- `delete(contacto)` (suspend)

6.4 RegistroSaludDao (RegistroSaludDao.kt)

- `getByUserId(userId)` → `Flow<List<RegistroSalud>>`
- `getById(id)` → `RegistroSalud?` (suspend)
- `insert(registro)` → `Long` (suspend)
- `delete(registro)` (suspend)

6.5 EventoCalendarioDao (EventoCalendarioDao.kt)

- `getByUserId(userId)` → `Flow<List<EventoCalendario>>`
- `getById(id)` → `EventoCalendario?` (suspend)
- `insert(evento)` → `Long` (suspend)
- `update(evento)` (suspend)
- `delete(evento)` (suspend)
- `deleteOlderThan(fecha)` (suspend)

6.6 MiembroFamiliarDao (MiembroFamiliarDao.kt)

- `getByMayorId(mayorId)` → `Flow<List<MiembroFamiliar>>`
- `getAllByMayorId(mayorId)` → `List<MiembroFamiliar>` (suspend)
- `getById(id)` → `MiembroFamiliar?` (suspend)
- `insert(miembro)` → `Long` (suspend)
- `update(miembro)` (suspend)
- `delete(miembro)` (suspend)

6.7 AbuelitoDao (AbuelitoDao.kt)

- `getAll()` → `Flow<List<Abuelito>>`
- `getByUserId(userId)` → `Flow<List<Abuelito>>`
- `getById(id)` → `Abuelito?` (suspend)
- `insert(abuelito)` → `Long` (suspend)
- `update(abuelito)` (suspend)
- `delete(abuelito)` (suspend)

6.8 ServicioHistorialDao (ServicioHistorialDao.kt)

- `getAll()` → `Flow<List<ServicioHistorial>>`
- `getById(userId)` → `Flow<List<ServicioHistorial>>`
- `insert(servicio)` → `Long` (suspend)

6.9 TrabajadorDao (`TrabajadorDao.kt`)

- `getAll()` → `Flow<List<Trabajador>>`
- `getAllList()` → `List<Trabajador>` — No reactivo, para Spinner
- `getById(id)` → `Trabajador?` (suspend)
- `insert(trabajador)` → `Long` (suspend)
- `update(trabajador)` (suspend)
- `delete(trabajador)` (suspend)
- `deleteAll()` (suspend)

7. Base de Datos (Room Database)

AppDatabase (`AppDatabase.kt`)

Clase abstracta que extiende `RoomDatabase` . Orquesta todas las entidades y DAOs.

Configuración:

- Nombre: `"helping_grandpa_db"`
- Versión: **4**
- Estrategia de migración: `fallbackToDestructiveMigration()`
- Patrón: **Singleton** con `getInstance(context)`

Entidades registradas:

- Trabajador, UserEntity, Abuelito, ServicioHistorial, Medicacion, ContactoEmergencia, RegistroSalud, EventoCalendario, MiembroFamiliar

8. Actividades (Pantallas)

8.1 MainActivity (`MainActivity.kt`)

Pantalla principal tipo hub con tarjetas de navegación a todas las funcionalidades.

Funcionamiento:

- Muestra splash screen al inicio
- Comprueba si debe mostrar onboarding (1ª vez)
- Solicita permisos de notificación POST_NOTIFICATIONS (Android 13+)
- Actualiza la UI según sesión iniciada/cerrada
- Navegación a: Servicio, Trabajo, Perfil/Registro, Medicación, Salud, Calendario, Familia, SOS
- Botones de Login, Registro, AcercaDe, Historial, Modo Desarrollador, Cerrar sesión

8.2 LoginActivity (`LoginActivity.kt`)

Inicio de sesión con email y password.

Flujo:

1. Validación de campos obligatorios
2. Autenticación local con `UserDao.login()` en hilo secundario

3. Si éxito: guarda sesión en `SessionManager`
4. Intento de login en Firebase Auth (si disponible)
5. Obtención de token FCM
6. Redirección a MainActivity

8.3 RegistroActivity (`RegistroActivity.kt`)

Registro de nuevo usuario.

Flujo:

1. Formulario con nombre, apellidos, email, password (mín 6 caracteres), teléfono, dirección
2. Validación de campos
3. Inserción en BD local (`UserDao.insert()`)
4. Si Firebase disponible: registro en Firebase Auth + obtención token FCM
5. Guardado de sesión y redirección

8.4 OnboardingActivity (`OnboardingActivity.kt`)

Tutorial de bienvenida en 4 slides con ViewPager2.

Slides:

1. Bienvenida a Helping a GrandPa!
2. Recordatorio de medicación
3. SOS emergencia
4. Familia y salud

Usa `SharedPreferences` para marcar como visto y no volver a mostrar.

8.5 PerfilActivity (`PerfilActivity.kt`)

Edición del perfil del usuario logueado. Carga datos desde Room y permite modificar nombre, apellidos, email, fecha de nacimiento (DatePicker), rol (Spinner). Guarda cambios con `UserDao.update()` .

8.6 ServicioActivity (`ServicioActivity.kt`)

Solicitud de servicios de asistencia (citas médicas y/o compras).

- Campos: horas médicas, fecha médica, horas compras, fecha compras
- Calcula precio (3.99 €/hora)
- Guarda en `ServicioHistorialDao.insert()`

8.7 TrabajoActivity (`TrabajoActivity.kt`)

Registro de trabajador/cuidador. Formulario con nombre, apellido, DNI/NIE, email, teléfono. Guarda en `TrabajadorDao.insert()` .

8.8 MedicacionActivity (`MedicacionActivity.kt`)

Lista de medicaciones del usuario.

- RecyclerView con `MedicacionAdapter`
- Click en tarjeta → editar medicación (pasa ID a `AgregarMedicacionActivity`)
- Botón eliminar → borra de BD y cancela WorkManager

8.9 AgregarMedicacionActivity (`AgregarMedicacionActivity.kt`)

Añadir/editar medicación.

- Campos: nombre, dosis, frecuencia, hora (TimePicker)
- Si recibe `medicacion_id` → modo edición (precarga datos)
- Guardado: insert o update según modo
- Programa recordatorio con `MedicacionWorkScheduler.programar()` (WorkManager)
- En edición: cancela WorkManager anterior y reprograma

8.10 SaludActivity (SaludActivity.kt)

Historial de salud del usuario.

- RecyclerView con `RegistroSaludAdapter`
- Botón "Añadir registro" → `AgregarSaludActivity`
- Botón "Exportar CSV" → `CsvExporter.exportarRegistrosSalud()`
- Botón "Ver gráficas" → `SaludGraficaActivity`
- Soporte de eliminación de registros

8.11 AgregarSaludActivity (AgregarSaludActivity.kt)

Nuevo registro de salud.

- Spinner para tipo (presión, glucosa, peso, ánimo)
- Campo valor, campo nota opcional
- Fecha generada automáticamente
- Guarda en `RegistroSaludDao.insert()`

8.12 SaludGraficaActivity (SaludGraficaActivity.kt)

Gráficas de salud con el componente personalizado `LineChartView` .

Filtra últimos 20 registros de cada tipo:

- **Presión arterial** → línea roja (valor sistólico)
- **Glucosa** → línea verde
- **Peso** → línea azul

8.13 SosActivity (SosActivity.kt)

Pantalla de emergencia SOS.

- Lista contactos con `ContactoEmergenciaAdapter`
- Botón "Añadir contacto" → `AgregarContactoActivity`
- Botón SOS:
 1. Solicita permisos (SMS, llamada, ubicación) si no concedidos
 2. Muestra diálogo de confirmación listando contactos
 3. Ejecuta `SosHelper.lanzarSos()` → SMS + WhatsApp + llamada + GPS

8.14 AgregarContactoActivity (AgregarContactoActivity.kt)

Añadir contacto de emergencia. Campos: nombre, teléfono, email. Guarda en `ContactoEmergenciaDao.insert()` .

8.15 FamiliaActivity (FamiliaActivity.kt)

Círculo familiar. Lista miembros con `MiembroFamiliarAdapter` . Soporte de eliminación local y remota (Firestore).

8.16 AgregarMiembroActivity (AgregarMiembroActivity.kt)

Invitar miembro familiar. Formulario con nombre, email, teléfono, rol (espectador, cuidador, admin). Guarda local y sincroniza a Firestore.

8.17 CalendarioActivity (CalendarioActivity.kt)

Calendario de eventos. Lista eventos con `EventoCalendarioAdapter` . Soporte de eliminación local y remota (Firestore).

8.18 AgregarEventoActivity (AgregarEventoActivity.kt)

Nuevo evento. Campos: título, descripción, fecha (DatePicker), hora (TimePicker), tipo (personal, médica, familiar, social). Guarda local y sincroniza a Firestore.

8.19 Otras Actividades

- **BaseDatosHelpingAgP** — Panel de administración "Oficina"
- **abuelosRegistrados** — Lista de mayores registrados
- **RegistrarAbuelitoActivity** — Registrar nueva persona mayor
- **HistorialActivity** — Historial de servicios solicitados
- **ListaDeRegistroTrab** — Búsqueda de trabajadores (Spinner + SearchView)
- **AcercaDeActivity** — Pantalla informativa

9. Adaptadores (RecyclerView)

Todos los adaptadores usan `ListAdapter` con `DiffUtil` para optimización de rendimiento, excepto dos inline que usan `RecyclerView.Adapter` clásico.

Adaptador	Layout	Elemento	Callbacks
MedicacionAdapter	item_medicacion.xml	Medicación	onEdit, onDelete
MiembroFamiliarAdapter	item_miembro_familia.xml	Miembro familiar	onDelete
EventoCalendarioAdapter	item_evento.xml	Evento	onDelete
ContactoEmergenciaAdapter	item_contacto_emergencia.xml	Contacto SOS	onDelete
RegistroSaludAdapter	item_registro_salud.xml	Registro salud	onDelete
OnboardingAdapter	item_onboarding_slide.xml	Slide onboarding	—
HistorialAdapter (inline)	item_historial.xml	Historial servicio	—
AbuelitoAdapter (inline)	item_abuelito.xml	Abuelito	—

10. Helpers y Utilidades

10.1 SessionManager (SessionManager.kt) — Singleton

Gestión de sesión de usuario mediante `SharedPreferences` .

Métodos:

- `init(context)` — Inicializa
- `saveSession(userEntity)` — Guarda datos de sesión
- `getUserId()` → Int
- `getUserName()` → String
- `getUserEmail()` → String
- `getUserRole()` → String
- `isLoggedIn()` → Boolean
- `logout()` — Limpia sesión y cierra Firebase Auth

10.2 NotificacionHelper (`NotificacionHelper.kt`) — Singleton

Gestión de notificaciones locales.

Canales:

- `medicacion` — Recordatorios de medicación
- `sos` — Alertas de emergencia
- `calendario` — Eventos de calendario

Método:

- `crearCanales(context)` — Crea canales en Android O+
- `mostrarNotificacion(context, channelId, titulo, mensaje)` — Lanza notificación

10.3 PermissionHelper (`PermissionHelper.kt`) — Singleton

Gestión de permisos runtime.

Permisos SOS: SEND_SMS, CALL_PHONE, ACCESS_FINE_LOCATION, ACCESS_BACKGROUND_LOCATION (Android 10+) **Permisos notificación:** POST_NOTIFICATIONS (Android 13+)

Métodos:

- `hasSosPermissions(context)` → Boolean
- `hasNotificationPermission(context)` → Boolean

10.4 SosHelper (`SosHelper.kt`) — Singleton

Ejecuta la secuencia de emergencia SOS.

Métodos:

- `lanzarSos(context, contactos)` — Obtiene ubicación, construye mensaje, envía a todos los contactos
- `enviarSms(context, telefono, mensaje)` — Envía SMS (o abre app si no hay permiso)
- `enviarWhatsApp(context, telefono, mensaje)` — Abre WhatsApp con mensaje predefinido
- `llamar(context, telefono)` — Inicia llamada (directa o dialer según permisos)
- `obtenerUbicacion(context, callback)` — Obtiene última ubicación con FusedLocationProviderClient

10.5 CsvExporter (`CsvExporter.kt`) — Singleton

Exportación de registros de salud a CSV.

Método:

- `exportarRegistrosSalud(context, registros)` — Genera CSV en Descargas, comparte con FileProvider

10.6 TrabajadorManager (TrabajadorManager.kt) — Singleton

Lista en memoria de trabajadores (para búsqueda rápida).

Métodos: agregar, registrar, filtrar, obtener lista.

10.7 LineChartView (LineChartView.kt) — Custom View

Componente de gráfica de línea dibujado en Canvas.

Propiedades:

- puntos: List<Pair<String, Float>> — Datos a graficar
- colorLinea: Int — Color de la línea

Características:

- Dibuja ejes, puntos, línea con trazo redondeado
 - Etiquetas de valores en el eje Y
 - Etiquetas de fecha en el eje X
 - Escala automática con margen del 10%
 - Texto "Sin datos" si lista vacía
-

11. Firebase

11.1 FirebaseManager (FirebaseManager.kt) — Singleton

Gestor central de Firebase.

Inicialización: init(context) — Inicializa FirebaseApp, Auth, Firestore **Flags:** disponible: Boolean — Indica si Firebase está operativo

Métodos:

- getAuth() → FirebaseAuth
- getFirestore() → FirebaseFirestore
- registrarUsuario(email, password) → FirebaseUser? (suspend)
- iniciarSesion(email, password) → FirebaseUser? (suspend)
- cerrarSesion() — Cierra sesión
- obtenerTokenFCM() → String (suspend)

11.2 FirestoreSyncHelper (FirestoreSyncHelper.kt) — Singleton

Sincronización bidireccional entre Room y Firestore.

Eventos de calendario:

- syncEvento(context, evento) — Sube evento a Firestore
- eliminarEventoRemoto(firestoreId) — Elimina evento remoto
- escucharEventos(context, userId) — SnapshotListener en tiempo real

Miembros familiares:

- syncMiembro(context, miembro) — Sube miembro a Firestore
- eliminarMiembroRemoto(firestoreId) — Elimina miembro remoto
- escucharFamilia(context, mayorId) — SnapshotListener en tiempo real

11.3 FCMSERVICE (`FCMSERVICE.kt`) — `FirebaseMessagingService`

Manejo de notificaciones push Firebase Cloud Messaging.

- `onNewToken(token)` — Guarda token en Firestore (colección `fcmtokens`)
 - `onMessageReceived(message)` — Procesa mensaje y muestra notificación local
 - Tipos: `medicacion` , `sos` , `calendario` — Cada uno usa su canal
 - `PendingIntent` abre `MainActivity`
-

12. Servicios en Segundo Plano

12.1 HelpingApp (`HelpingApp.kt`) — `Application`

Clase `Application` personalizada. Se ejecuta al arrancar la app.

Inicializaciones:

1. `SessionManager.init(this)`
2. `NotificacionHelper.crearCanales(this)`
3. `FirestoreManager.init(this)`
4. `FirestoreSyncHelper.init(FirestoreManager.getFirestore())`

12.2 MedicacionWorkScheduler (`MedicacionWorkScheduler.kt`) — `Singleton`

Programador de recordatorios de medicación con `WorkManager`.

Métodos:

- `programar(context, medicacionId, nombre, dosis, hora)` — Calcula delay hasta la hora, crea `OneTimeWorkRequest` con tag único
- `cancelar(context, medicacionId)` — Cancela el trabajo por ID
- `reprogramarTrasNotificacion(context, ...)` — Reprograma para el día siguiente

12.3 MedicacionWorker (`MedicacionWorker.kt`) — `CoroutineWorker`

Worker que ejecuta el recordatorio de medicación.

`doWork()`:

1. Extrae nombre, dosis, ID de `inputData`
2. Verifica en BD que la medicación siga activa
3. Si activa: muestra notificación
4. No reprograma automáticamente (la activity lo hace al guardar)

12.4 AlarmaMedicacionReceiver (`AlarmaMedicacionReceiver.kt`) — `BroadcastReceiver`

`BroadcastReceiver` alternativo usando `AlarmManager` (legacy).

Métodos:

- `onReceive(context, intent)` — Muestra notificación
- `crearPendingIntent(context, medicacionId, nombre, dosis)` — Companion, genera `PendingIntent`

12.5 BootReceiver (`BootReceiver.kt`) — `BroadcastReceiver`

Receptor de `ACTION_BOOT_COMPLETED` .

Al reiniciar el dispositivo:

1. Obtiene todos los usuarios de la BD
 2. Para cada usuario, obtiene medicaciones activas
 3. Reprograma cada medicación con `MedicacionWorkScheduler.programar()`
-

13. UI y Accesibilidad

Temas y Estilos (`themes.xml`)

La interfaz está diseñada específicamente para personas mayores:

Tipografía adaptada:

- `HeadlineLarge`: 36sp, bold
- `HeadlineMedium`: 30sp, bold
- `TitleMedium`: 22sp, bold
- `BodyLarge`: 18sp
- `BodyMedium`: 16sp
- `LabelLarge`: 15sp, semibold

Componentes:

- `AccesibleButton` : `minHeight 64dp`, `textSize 20sp`, negrita
- `EmergencyButton` (SOS): `minHeight 80dp`, `textSize 24sp`, mayúsculas, color rojo
- `PremiumCard` : `border-radius 16dp`, `padding 24dp`
- Actividades con animaciones `slide_in/slide_out`

Colores de alto contraste:

- `on_surface` : `#000000` (negro puro)
- `primary` : `#0055CC` (azul oscuro)
- `error` : `#D92D20` (rojo intenso)
- `btn_emergency` : `#D92D20`

Iconos vectoriales

Se han reemplazado todos los emojis por iconos vectoriales Material Design para un aspecto más profesional:

- `ic_medicacion.xml` — Pastilla
 - `ic_salud.xml` — Corazón
 - `ic_calendario.xml` — Calendario
 - `ic_familia.xml` — Grupo de personas
 - `ic_servicio.xml` — Edificio
 - `ic_trabajo.xml` — Maletín
 - `ic_perfil.xml` — Persona
-

14. Tests

Tests Unitarios (10) — `ModelUnitTest.kt` + `CsvExporterUnitTest.kt`

Ubicados en `app/src/test/java/com/example/miaplicacion/`

`ModelUnitTest` (10 tests):

- Creación de entidades `Medicacion` , `ContactoEmergencia` , `RegistroSalud`
- Verificación de valores por defecto (activo=true, id=0)
- Formato de fecha en `RegistroSalud`
- Data class `EventoCalendario` con tipos
- Data class `MiembroFamiliar` con roles
- Pruebas de `CsvExporter` (formato CSV, cabeceras)

Tests Instrumentados (10) — `DatabaseInstrumentedTest.kt`

Ubicados en `app/src/androidTest/java/com/example/miaplicacion/`

- CRUD completo de `MedicacionDao`
- CRUD completo de `ContactoEmergenciaDao`
- CRUD completo de `RegistroSaludDao`
- CRUD completo de `EventoCalendarioDao`
- CRUD completo de `MiembroFamiliarDao`
- Pruebas de Flow (reactividad)
- Filtrado por `userId`

15. Landing Page

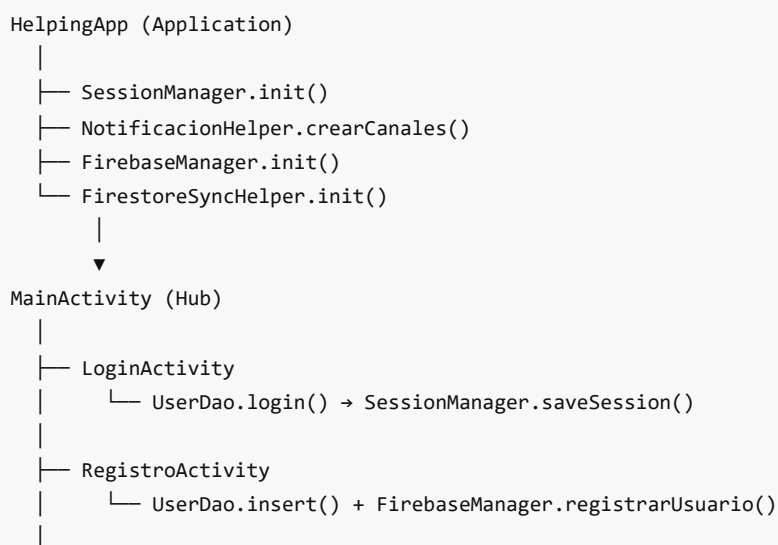
Se ha desarrollado una **landing page promocional** independiente con React + Vite para presentar el proyecto.

Ubicación: `D:\DAM\SegundoDam\TrabajoFinalDeGrado\landing-page\`

Características:

- Diseño moderno tipo Apple con animaciones CSS
- Iconos SVG personalizados para cada funcionalidad
- Imágenes de Unsplash relacionadas con cuidado de mayores
- Animaciones de entrada al hacer scroll (Intersection Observer)
- Diseño responsive (adaptado a móvil y escritorio)

16. Diagrama de Interacciones



- └─ MedicacionActivity
 - └─ AgregarMedicacionActivity → MedicacionDao.insert/update
 - └─ MedicacionWorkScheduler.programar() → MedicacionWorker
 - └─ BootReceiver → Reprograma en reinicio
- └─ SaludActivity
 - └─ AgregarSaludActivity → RegistroSaludDao.insert()
 - └─ SaludGraficaActivity → LineChartView
 - └─ CsvExporter.exportarRegistrosSalud()
- └─ SosActivity
 - └─ AgregarContactoActivity → ContactoEmergenciaDao.insert()
 - └─ PermissionHelper.hasSosPermissions()
 - └─ SosHelper.lanzarSos() → SMS + WhatsApp + llamada + GPS
- └─ CalendarioActivity
 - └─ AgregarEventoActivity → EventoCalendarioDao.insert()
 - └─ FirestoreSyncHelper.syncEvento() → Firestore
- └─ FamiliaActivity
 - └─ AgregarMiembroActivity → MiembroFamiliarDao.insert()
 - └─ FirestoreSyncHelper.syncMiembro() → Firestore
- └─ FCMService → Notificaciones push cloud

Estadísticas del Proyecto

Métrica	Valor
Archivos Kotlin	62
Actividades	24
Room Entities	9
Room DAOs	9
Adapters	8
Helpers/Utilities	7
Firebase-related	3
Background Services	5
Layouts XML	32
Tests unitarios	10
Tests instrumentados	10
minSdk	27 (Android 8.1 Oreo)
targetSdk	36

Documentación generada el 15 de mayo de 2026

Helping a GrandPa! — TFG Desarrollo de Aplicaciones Multiplataforma